LAB 3: ADC, DAC and DMA

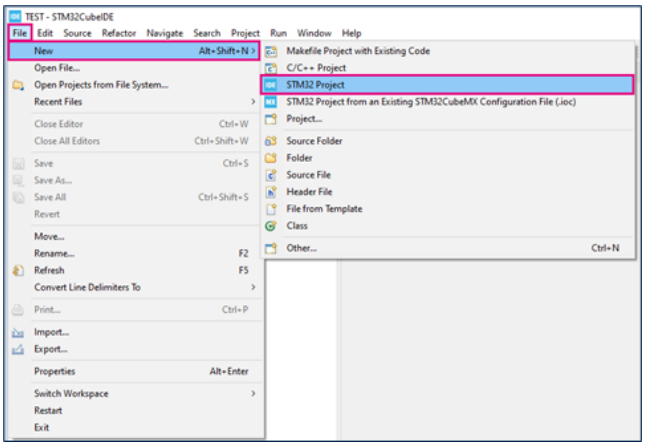
Part 1: Configure ADC to measure the DAC output

2.1. Objectives

- Learn how to set up the ADC with DMA in STM32CubeMX.
- Generate code in STM32CubeMX and using HAL functions.
- Create a simple application to start the ADC and measure the DAC output.

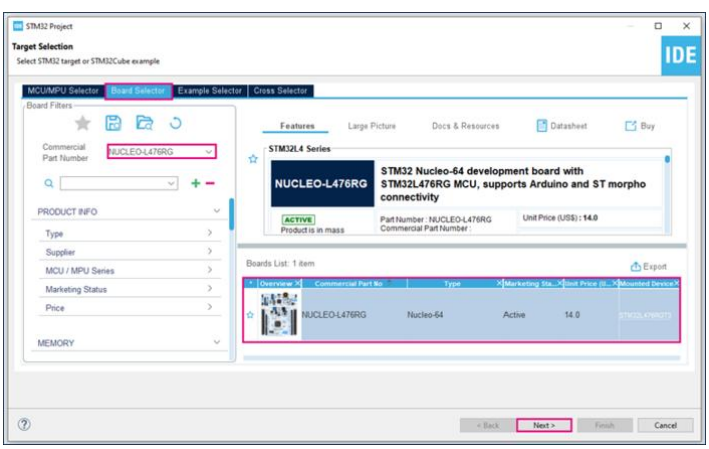
2.2. Create the project in STM32CubeIDE

- **File** > **New** > **STM32 Project** in main panel.

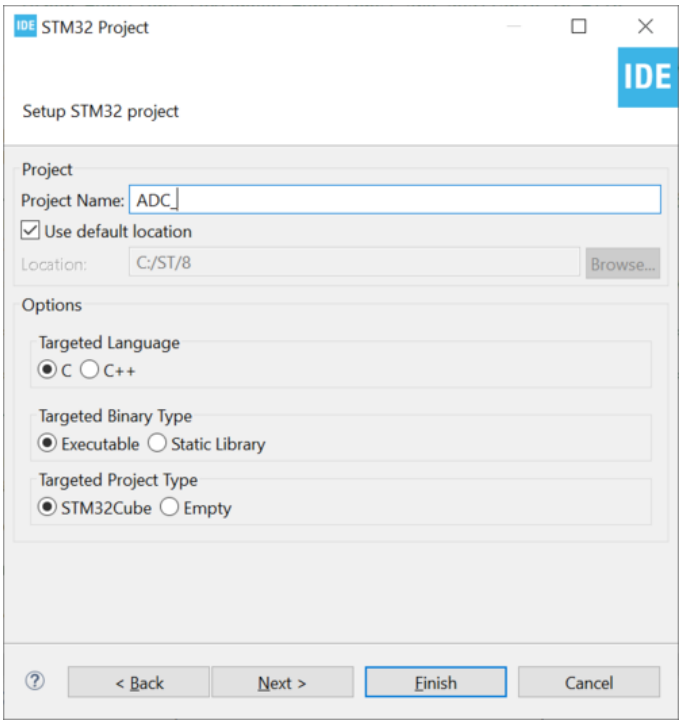


This example uses the NUCLEO-L476RG board.

- Select **NUCLEO-L476RG** board using **Board selector** as shown in the figure below:



- Save the project:

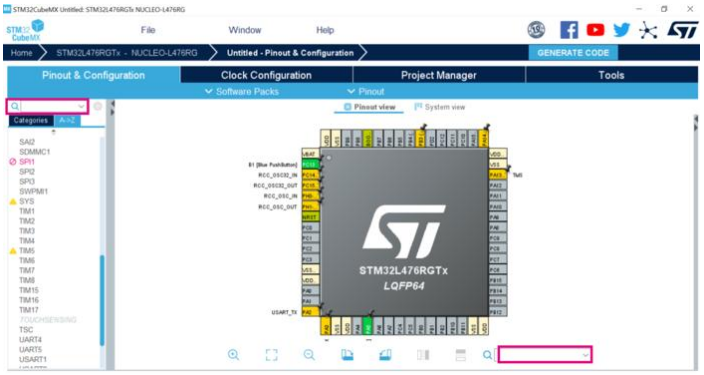


2.1. Configure peripherals

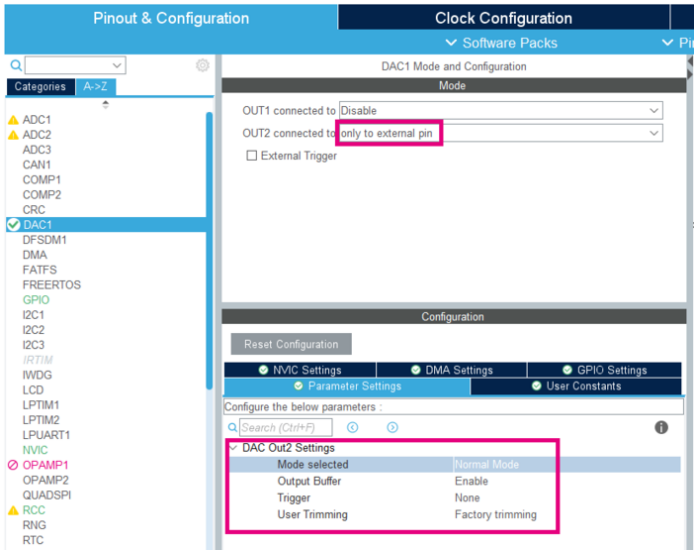
2.1.1. Configure DAC

We will use **PA5** as the DAC's output. As it is configured as GPIO Output by default, we need to reset it first.

- Search for **PA5** in the bottom right search field, then click **Reset_State**.



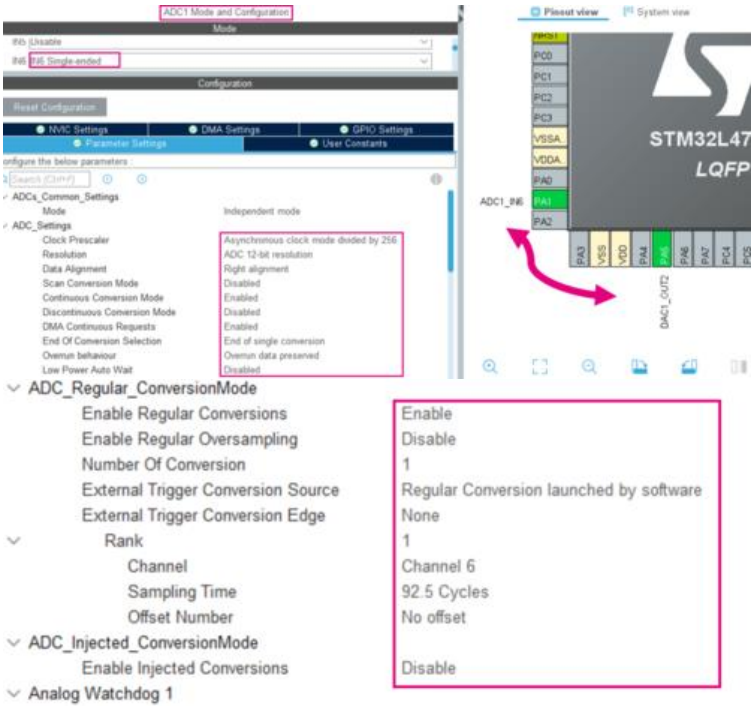
- Search for **DAC1** in the left search field.
- OUT2 connected to **only to external pin**. Notice how **PA5** is automatically set to DAC1_OUT2 on the pinout view.
- **Parameter Settings** should be set up as in the below example: **normal mode**, **buffer enabled** and **no trigger**.



2.1.2. Configure ADC

Configure ADC with the following settings:

- Asynchronous clock mode div by **256**
- Resolution 12bits
- Right alignment
- Continuous conversion mode **Enabled**
- Software trigger
- sampling time for CH6 (92.5cycles)
- DMA Continuous requests **Enabled**

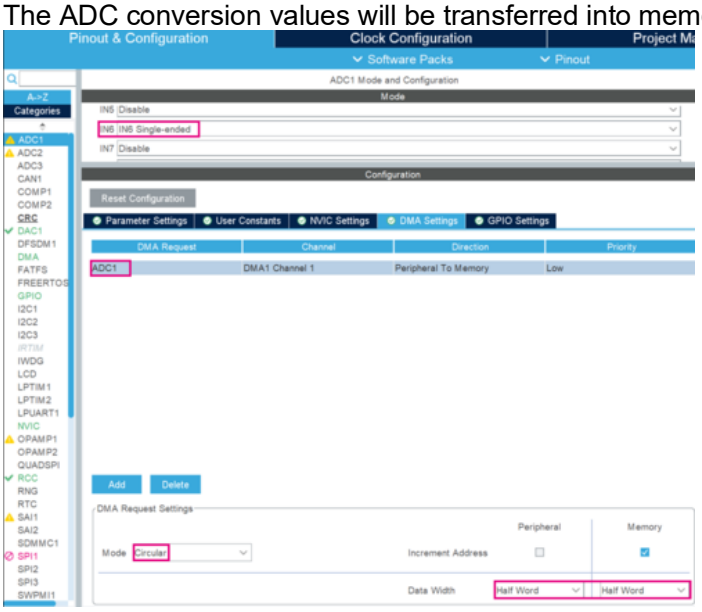


Information

Make sure to connect PA1 with PA5. You can use a jumper for that.

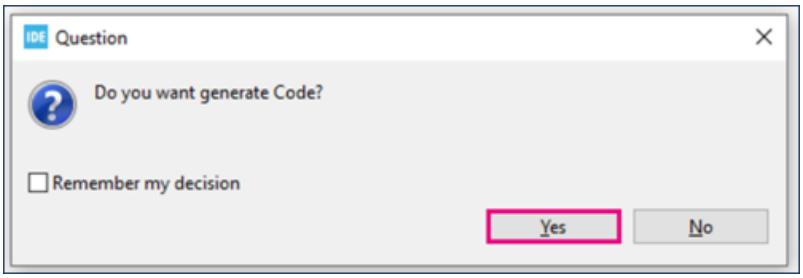
2.1.3. Configure DMA

The ADC conversion values will be transferred into memory by DMA controller.



2.2. Generate the project and edit main.c

- Click **Ctrl+S** to generate the project:



- Edit **main.c**:

Information

Insert your code between /* USER CODE BEGIN PV */ and /* USER CODE END PV */ tags

```
/* USER CODE BEGIN PV */
```

```
uint32_t value_adc;
uint32_t value_dac=0;
/* USER CODE END PV */
```

Information

Start the DAC, the ADC calibration and the ADC conversion by inserting the required code between /* USER CODE BEGIN 2*/ and /* USER CODE END 2 */ tags

```
/* USER CODE BEGIN 2 */
HAL_DAC_Start(&hdac1,DAC_CHANNEL_2);
HAL_ADCEx_Calibration_Start(&hadc1, ADC_SINGLE_ENDED);
HAL_ADC_Start_DMA(&hadc1,(uint32_t*)&value_adc,1);
/* USER CODE END 2 */
```

Information

Set the DAC output value by inserting the required code between /* USER CODE BEGIN 3 */ and /* USER CODE END 3 */ tags

```
/* USER CODE BEGIN 3 */
while (1)
{
  HAL_DAC_SetValue(&hdac1, DAC_CHANNEL_2, DAC_ALIGN_12B_R, value_dac);
  value_dac++;
  if(value_dac>4095) {
    value_dac=0;
  }
}
/* USER CODE END 3 */
```

- ***Explanation***

First, DAC is configured as input voltage source for ADC. It uses output channel 2 (OUT2) and is connected to an external pin (PA5).
The ADC is configured to use asynchronous clock mode with a clock prescaler of 256 ensuring that ADC is not running too fast.
ADC is configured to have a resolution of 12 bits, right alignment, continuous conversion mode enabled (in order to convert data continuously), and software trigger with a sampling time of 92.5 cycles for channel 6 (PA1).
The ADC values are transferred to memory using DMA controller, which is configured as follows :

- Circular mode, allowing an updated indefinitely transfer.
- Continuous DMA request enabled allowing an uninterrupted transfer of data to the memory.

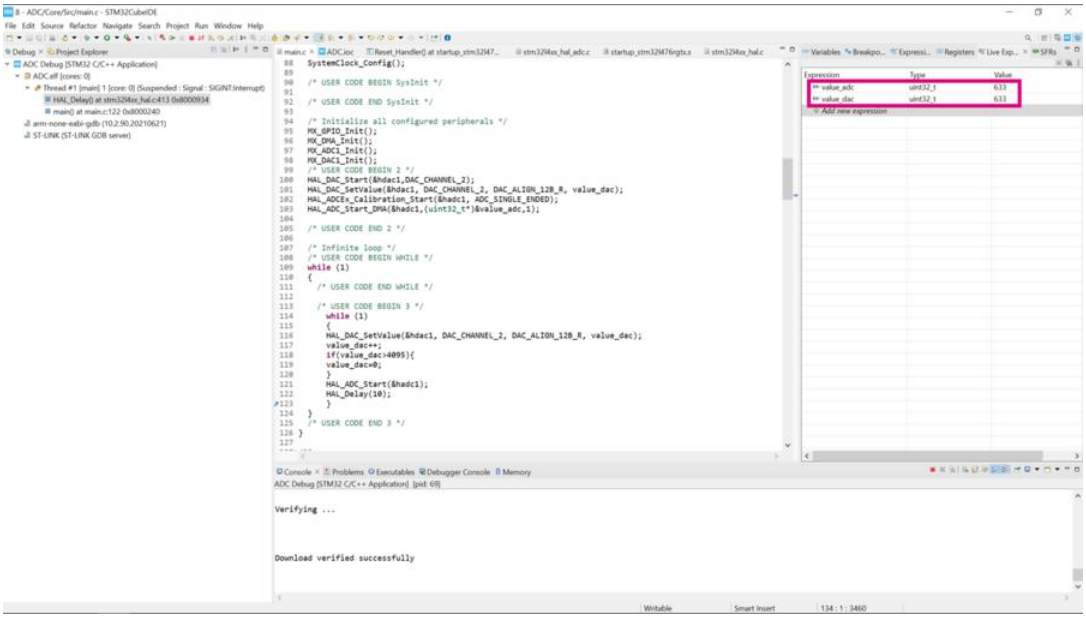
ADC and DAC should have **similar** values, while using PA1 – PA5 jumper.
Therefore, the code will sweep through all the possible ADC output values, and then reset it to zero restarting the process all over again.

2.3. Compile and flash

- Click the **Build** button
- Click the **Debug** button to run it step by step

3. Build and measure a simple signal generator

- Put jumper on PA1 - PA5 pins to connect DAC output to ADC input. The result is as follows



Part 2:

Objectives

- Test the analog temperature sensor.
- Test the touch Sensor Module for both analog and digital values.
- Test the Joystick modules.

Deliverables

- Code.
- Video showing the output
- Lab report showing your work.